Scaling up RL

10-701 Introduction to Machine Learning

Geoff Gordon

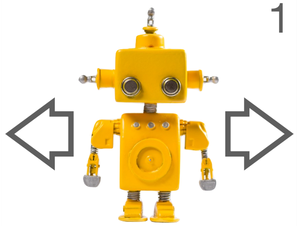
Advantage

- Def'n: ***advantage function*** is $A(s,a) = Q(s,a) - V(s)$
  - with A^π and A^* versions using Q^π, V^π and Q^*, V^*
    - A^*: how does action a compare to optimal action in s?
    - A^π: how does a compare to the distribution $\pi(a \mid s)$?
  - if $A(s,a) \geq 0$ then $Q(s,a) \geq V(s)$: a is at least as good
    - else $Q(s,a) < V(s)$: a is worse
- Intuitively, $A^\pi(s,a)$ seems related to improving π
- *Note:* still call it advantage even if we're using costs
  - "disadvantage" would be more accurate

Advantage example

$\gamma = 1$

Environment

1	2	3	4	5
				
r=–1	r=–1	r=–10	r=–1	r=0

V^*

1	2	3	4	5
–13	–12	–11	–1	0

Q^*

	1	2	3	4	5
L	–14	–14	–22	–12	0
R	–13	–12	–11	–1	0

A^*

	1	2	3	4	5
L	–1	–2	–11	–11	0
R	0	0	0	0	0

Geoff Gordon

State visitation frequency

- Define the ***visitation frequency*** for π:

$$d^\pi(s) = \mathbb{E}[\mathbb{I}(s_1 = s) + \gamma \mathbb{I}(s_2 = s) + \gamma^2 \mathbb{I}(s_3 = s) + \ldots]$$

  - where $\mathbb{I}(s_t = s)$ is 1 if we visit state s at time t
  - and zero o/w, including if episode ends before time t

- An unnormalized probability distribution
  - sum is the expected episode length for undiscounted case, or $\frac{1}{1-\gamma}$ for discounted infinite horizon

- We can write the total reward of π (what we want to optimize) as

$$J = \sum_{s,a} d^\pi(s)\, \pi(a \mid s)\, r(s,a)$$

Performance difference lemma

like expression for J but with advantage instead of reward

- The ***performance difference lemma*** formalizes the idea that $A^{\pi}(s, a)$ is related to improving our policy
- For any policies π_1 and π_2 with total rewards J_1 and J_2

$$J_1 - J_2 = \sum_{s,a} d^{\pi_1}(s)\, \pi_1(a \mid s)\, A^{\pi_2}(s, a)$$

  - to get change in total reward, visit s, a according to π_1's frequencies and evaluate A^{π_2}
- Proof sketch: $A^{\pi_2}(s, a)$ tells us how much we win each time we choose action a in s (instead of following π_2)
  - the sum is how much we win in total while following π_1

Policy gradient

follows from performance difference lemma as $\pi_1 \to \pi_2$

- ***Thm**:* the gradient(*) of total reward J wrt policy $\pi(a \mid s)$ is

$$g^\pi(s,a) = d^\pi(s)\, A^\pi(s,a) \quad \text{[dimension } |\mathcal{S}| \times |\mathcal{A}|\text{]}$$

- To improve policy quickly, increase $\pi(a \mid s)$ if
  - action a is better than alternatives (large $A^\pi(s,a)$)
  - state s is visited often (large $d^\pi(s)$)
- Expression above is gradient wrt π itself (tabular case)
  - if $\pi_\theta(a \mid s)$ is a learned function with parameters θ then

$$\frac{dJ}{d\theta} = \sum_{s,a} g(s,a) \frac{d}{d\theta} \pi_\theta(a \mid s) \quad \text{(chain rule)}$$

(*) Because of constraint $\sum_a \pi(a \mid s) = 1 \ (\forall s)$, gradient isn't uniquely defined. Can add any per-state constant, $g(s,a) + b(s)$, to get a valid gradient.

Stochastic gradient estimates

- Previous slide gave the exact gradient $\frac{dJ}{d\pi}$ or $\frac{dJ}{d\theta}$
  - but it's too expensive to use (have to iterate over all s, a)
- Instead, need a stochastic estimate that we can use with SGD-like updates in policy gradient
  - need correct expected value, but sampling noise is OK
- Three common ways to get such estimates
  - actor-critic
  - REINFORCE
  - reparameterization

Sampling states

- From above: with parameterized policy $\pi_\theta(a \mid s)$,

$$\frac{dJ}{d\theta} = \sum_{s,a} d^\pi(s)\, A^\pi(s,a) \frac{d}{d\theta} \pi_\theta(a \mid s)$$

- RHS looks like an (unnormalized) expectation:
  - sample a state s according to (normalized) $d^\pi(s)$
  - compute $\hat{g}(\theta) = \sum_a A^\pi(s,a) \frac{d}{d\theta} \pi_\theta(a \mid s)$, so that

$$\mathbb{E}(\hat{g}(\theta)) = \sum_s \frac{1}{Z} d^\pi(s) \sum_a A^\pi(s,a) \frac{d}{d\theta} \pi_\theta(a \mid s) = \frac{1}{Z} \frac{dJ}{d\theta}$$

    where Z is normalizer for d^π
  - as desired!

Actor-critic

- Don't know A^π?
  - learn Q^π, V^π with methods above!
  - called a ***critic*** (in contrast with the ***actor***, π)
- How do we sample from d^π / Z?
  - run a trajectory according to π
  - keep state from step t with probability $\propto \gamma^{t-1}$
  - or just keep all states and weight by γ^{t-1}
  - or don't even bother with weighting (wrong but useful)
- With above answers, we get ***actor-critic*** method
- Worry: d^π is easy, but what if our estimate of A^π is bad? Can we avoid dependence on this estimate?

Sampling actions as well as states

- Again, $\frac{dJ}{d\theta} = \sum_{s,a} d^\pi(s)\, A^\pi(s,a) \frac{d}{d\theta} \pi_\theta(a \mid s)$
- Useful fact:
  - $\frac{d}{d\theta} \ln \pi_\theta(a \mid s) = \frac{1}{\pi_\theta(a \mid s)} \frac{d}{d\theta} \pi_\theta(a \mid s)$ (chain rule)
  - so $\pi_\theta(a \mid s) \frac{d}{d\theta} \ln \pi_\theta(a \mid s) = \frac{d}{d\theta} \pi_\theta(a \mid s)$
- Substituting:

$$\frac{dJ}{d\theta} = \sum_{s,a} d^\pi(s)\, \pi_\theta(a \mid s)\, A^\pi(s,a) \frac{d}{d\theta} \ln \pi_\theta(a \mid s)$$

- This looks like an (unnormalized) expectation where we sample s, a proportional to $d^\pi(s)\, \pi_\theta(a \mid s)$
  - i.e., just sample a step s, a from a trajectory following π_θ

Advantage estimates

- Means we only need advantage estimates for s, a we actually executed during a trajectory
  - i.e., we need estimates of $Q^\pi(s, a)$ and $V^\pi(s)$
- $Q^\pi(s, a)$ is easy: sum of rewards starting from s, a
  - unbiased! (but higher variance)
- Turns out we can use *any* estimate of $V^\pi(s)$
  - $\sum_a \pi_\theta(a \mid s) = 1$
  - differentiate both sides: $\sum_a \frac{d}{d\theta} \pi_\theta(a \mid s) = 0$
  - use log identity: $\mathbb{E}_{a \sim \pi} \frac{d}{d\theta} \ln \pi_\theta(a \mid s) = 0$
  - so any per-state offset to A^π cancels out on average
  - called a ***baseline*** in this context

Policy gradient theorem

d = number of parameters in policy, so $\theta \in \mathbb{R}^d$

showing undiscounted finite horizon case (but others work too)

$$J(\theta) = \mathbb{E}_{\pi_\theta}[r_1 + r_2 + \ldots + r_T]$$

- On iteration m, get trajectory $\tau^m = (s_1^m, a_1^m, r_1^m \ldots, s_T^m, a_T^m, r_T^m)$ by following policy $\pi_\theta(a_t^m \mid s_t^m)$
- Define

$A_t^m = \sum_{i=t}^T r_i^m \in \mathbb{R}$ (empirical total reward starting from step t) — optionally subtract baseline estimate of $V^\pi(s_t^m)$

$u_t^m = \frac{d}{d\theta} \ln \pi_\theta(a_t^m \mid s_t^m) \in \mathbb{R}^d$ (*action score vector,* from autodiff)

$g^m = \sum_{t=1}^T A_t^m u_t^m \in \mathbb{R}^d$ (the gradient estimate)

Theorem:

g^m is an unbiased estimate of $\frac{d}{d\theta} J(\theta)$

Policy gradient theorem

d = number of parameters in policy, so $\theta \in \mathbb{R}^d$

showing undiscounted finite horizon case (but others work too)

$$J(\theta) = \mathbb{E}_{\pi_\theta}[r_1 + r_2 + \ldots + r_T]$$

- On iteration m, get trajectory $\tau^m = (s_1^m, a_1^m, r_1^m \ldots, s_T^m, a_T^m, r_T^m)$ by following policy $\pi_\theta(a_t^m \mid s_t^m)$
- Define

$A_t^m = \sum_{i=t}^{T} r_i^m \in \mathbb{R}$ (empirical total reward starting from step t) — optionally subtract baseline estimate of $V^\pi(s_t^m)$

$u_t^m = \frac{d}{d\theta} \ln \pi_\theta(a_t^m \mid s_t^m) \in \mathbb{R}^d$ (*action score vector,* from autodiff)

$g^m = \sum_{t=1}^{T} A_t^m u_t^m \in \mathbb{R}^d$ (the gradient estimate)

Theorem:

g^m is an unbiased estimate of $\frac{d}{d\theta} J(\theta)$

even if we don't know anything about the environment

Policy gradient theorem

d = number of parameters in policy, so $\theta \in \mathbb{R}^d$

showing undiscounted finite horizon case (but others work too)

$$J(\theta) = \mathbb{E}_{\pi_\theta}[r_1 + r_2 + \ldots + r_T]$$

- On iteration m, get trajectory $\tau^m = (s_1^m, a_1^m, r_1^m \ldots, s_T^m, a_T^m, r_T^m)$ by following policy $\pi_\theta(a_t^m \mid s_t^m)$
- Define

$A_t^m = \sum_{i=t}^T r_i^m \in \mathbb{R}$ (empirical total reward starting from step t) — optionally subtract baseline estimate of $V^\pi(s_t^m)$

$u_t^m = \frac{d}{d\theta} \ln \pi_\theta(a_t^m \mid s_t^m) \in \mathbb{R}^d$ (*action score vector,* from autodiff)

$g^m = \sum_{t=1}^T A_t^m u_t^m \in \mathbb{R}^d$ (the gradient estimate)

Theorem:

g^m is an unbiased estimate of $\frac{d}{d\theta} J(\theta)$

even if we don't know anything about the environment

and ***even if*** environment is PO (w/ o_t^m instead of s_t^m)

Policy gradient intuition

- From above, gradient estimate $g^m = \sum_{t=1}^{T} A_t^m u_t^m$
- Score vectors u_t^m: parameter direction that would increase (log-)probability of taking action a_t^m in state s_t^m
- Scale by A_t^m: upweight score vector when advantage is large, flip score vector if advantage is negative
- On average: a direction that changes policy by taking actions more often if they were associated with high (total future) rewards
  - step along this direction: due to the log, change policy *multiplicatively* in favor of high-reward actions
- To minimize costs, step along *negative* gradient: take actions *less* often if associated with high costs

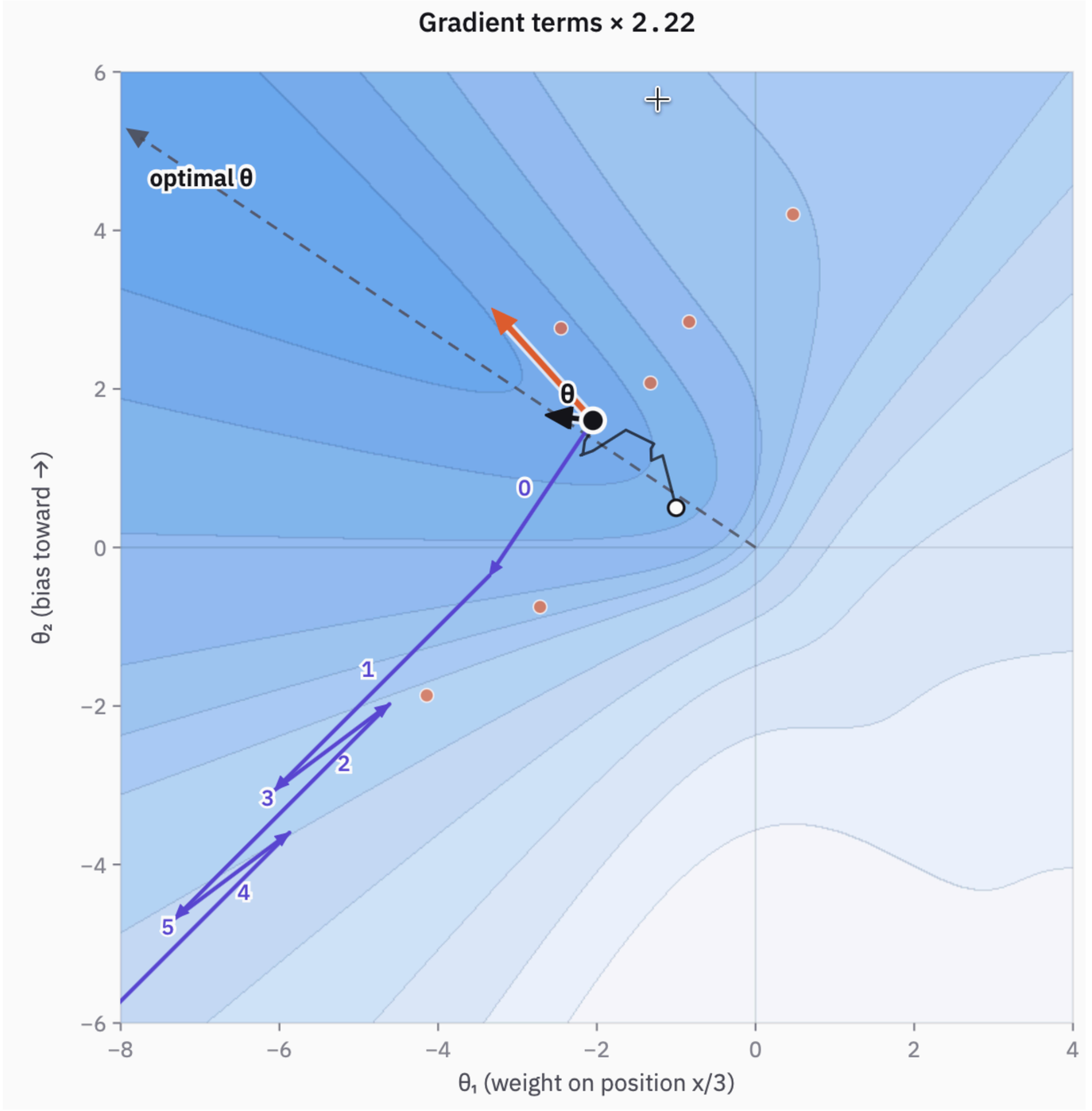
Discounting for policy gradient estimate

- With discounting, estimate is instead
  - $g^m = \sum_{t=1}^{T} \gamma^{t-1} A_t^m u_t^m$ where $A_t^m = \sum_{i=t}^{T} \gamma^{i-t} r_i^m$
  - or $g^m = \sum_{t=1}^{T} \sum_{i=t}^{T} \gamma^{i-1} r_i^m u_t^m$
  - one term for every pair of timesteps (t, i) with $i \geq t$
- Problem: many of these terms have very low weight
  - we ignore what happens after γ^{i-1} becomes small
- For this reason, we often drop the γ^{t-1} term in the first equation above (same wrong-but-useful idea as before)
  - allows us to learn from the end of a long trajectory
  - biases gradient estimate — in an empirically benign way

REINFORCE

- If we plug the estimate from the policy gradient theorem into the policy gradient method, we get *REINFORCE* [Williams, 1992], one of the oldest RL algorithms
- Repeat:
  - gather some trajectories under current policy π_θ
  - compute gradient estimate g by formula above
  - update θ by SGD

REINFORCE example



Continuous actions

- For continuous actions, policy is often given as a network whose outputs are parameters of conditional distribution over actions
  - $\pi_\theta(a \mid s) = \mathcal{N}(a; \mu_\theta(s), \sigma^2)$
  - take σ^2 as fixed for now (can generalize)
- We can ***reparameterize*** to isolate randomness
  - $a = \mu_\theta(s) + z$ where $z \sim \mathcal{N}(0, \sigma^2)$

Continuous actions

- Writing $A_s(a)$ for the advantage for each state s (considered as a function over a, $A^\pi(s, \cdot)$) we can rearrange the REINFORCE gradient formula:

$$\frac{dJ}{d\theta} = \sum_s d^\pi(s) \int_a \pi_\theta(a \mid s)\, A_s(a)\, \frac{d}{d\theta} \ln \pi_\theta(a \mid s)\, da$$

$$= \sum_s d^\pi(s) \int_a A_s(a) \frac{d}{d\theta} \pi_\theta(a \mid s)\, da$$

$$= \sum_s d^\pi(s) \frac{d}{d\theta} \int_a A_s(a)\, \pi_\theta(a \mid s)\, da$$

- Substituting $a = \mu_\theta(s) + z$

$$\frac{dJ}{d\theta} = \sum_s d^\pi(s) \frac{d}{d\theta} \int_z A_s(\mu_\theta(s) + z)\, \mathcal{N}(z)\, dz$$

$$= \sum_s d^\pi(s) \int_z A'_s(\mu_\theta(s) + z) \frac{d}{d\theta} \mu_\theta(s)\, \mathcal{N}(z)\, dz$$

Reparameterization gradient

$$\frac{dJ}{d\theta} \sum_s d^\pi(s) \int_z A'_s(\mu_\theta(s) + z) \frac{d}{d\theta} \mu_\theta(s) \, \mathcal{N}(z) \, dz$$

- That is, we can get gradient samples by taking s, z from a trajectory (note: randomness z instead of action a)
  - we need $A'_s(a) = \frac{d}{da} A^\pi(s, a)$ — not used in other methods we showed above
- We typically substitute a learned estimate of A^π
  - adds bias
  - have to hope our learned advantage estimate has accurate derivatives wrt actions

Example: AlphaGo

- Training ran on cluster of 50 GPUs; play used even more
- Training:
  - supervised policy: minibatches of 16 positions, 340,000,000 iterations of async SGD via parameter server, ~1k GPU-days
  - REINFORCE: minibatches of 128 games, embarrassingly parallel; 10,000 iterations of synchronous policy gradient, 50 GPU-days
  - self-play data: generated 30,000,000 positions, each from a separate game, embarrassingly parallel
  - Value network: minibatches of 32 positions, 50,000,000 iterations of async SGD, parameter server, 350 GPU-days
- Play: custom parallel variant of MCTS (Monte-Carlo tree search, a way to look ahead in the game tree)

Extensions

- Better advantage estimates:
  - GAE: mix rollouts and value critic to trade bias/variance
  - DAE: in expectation, sum of (reward – advantage) along a trajectory = V(start) – V(end); use this constraint to help estimate A^π
  - RLOO, GRPO: use multiple trajectories in a minibatch as baselines to reduce each other's variance
- Complete systems including additional variance reduction ideas plus how to manage sampling vs. optimization
  - PPO (extends REINFORCE, aspects of actor-critic)
  - SAC (extends reparameterization, aspects of actor-critic)
- Pairwise preference terminal rewards:
  - DPO to reduce RL to classification

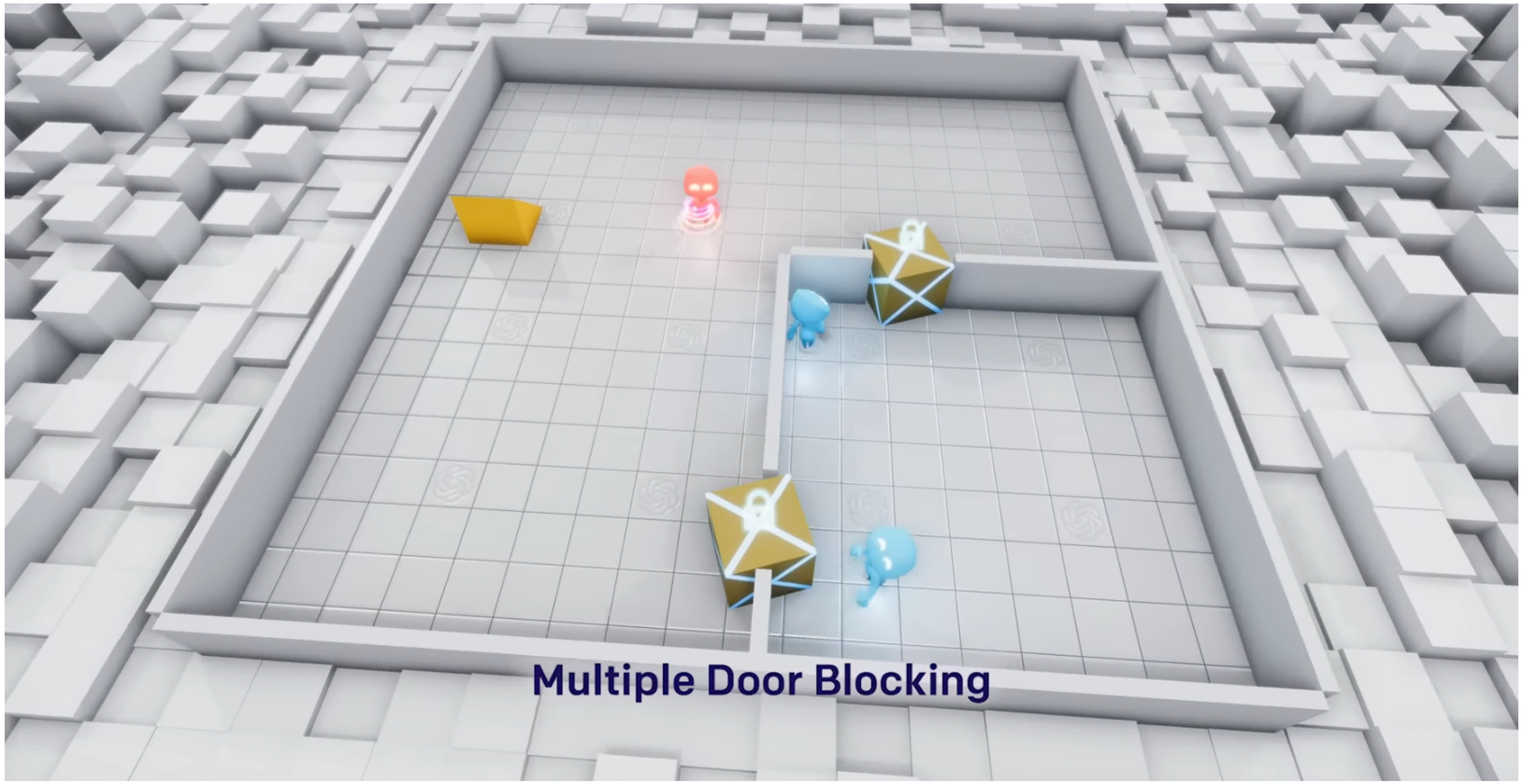
Learning objectives—RL

- What information is included in an environment model? Describe examples like MDP, POMDP, contextual bandits, control
- What makes RL hard?
- What are value, Q, and advantage functions (both * and π variants)? What is a greedy policy?
- Write and explain the Bellman equations for different cases (with or w/o max, exact or sampled, discounted or undiscounted)
- How should we handle termination and truncation?

Learning objectives—RL

- What issues do we encounter when scaling up to environments that are too big for tabular RL?
- Implement RL methods: value/Q iteration, TD, SARSA, REINFORCE, actor-critic
- Explain the performance difference lemma, policy gradient, and policy gradient theorem
- Use the reparameterization trick to get gradients in a continuous-action problem

Another fun example



- OpenAI project using large-scale RL and self-play [Baker et al., 2020]

Learning theory

10-701 Introduction to Machine Learning
Geoff Gordon

with thanks to Matt Gormley and Henry Chai

Statistical learning theory setup

1. Data points are generated i.i.d. from some *unknown* distribution $\boldsymbol{x}^{(n)} \sim p^*(\boldsymbol{x})$ and added to training dataset X
2. Labels are generated from some *unknown* function $y^{(n)} = c^*\left(\boldsymbol{x}^{(n)}\right) \in \{-1, +1\}$ and added to Y

   note: ***binary*** *classification*

   - maybe with some label noise: flip $y^{(n)}$ w.p. η
3. Learning algorithm chooses a hypothesis (classifier) with low *training* error rate from hypothesis set, $\mathscr{H}$
4. Goal: return a hypothesis (classifier) with low *true* error rate (imagine measuring on very large *test* set)

Types of Error

- True error rate
  - Actual quantity of interest for learning
  - How well your hypothesis will perform on new samples
- Training error rate
  - Used to choose $h \in \mathscr{H}$ (e.g., fit model parameters)
  - May be a very optimistic estimate of true error
- Test error rate
  - Used to evaluate hypothesis performance
  - Good estimate of true error (w/ enough test data)
- Validation error rate
  - Used to help choose $\mathscr{H}$ (e.g., set hyperparameters)
  - Somewhat optimistic estimate of true error

Types of Error

- True error rate
  - Actual quantity of interest for learning
  - How well your hypothesis will perform on new samples
- Training error rate
  - Used to choose $h \in \mathcal{H}$ (e.g., fit model parameters)
  - May be a very optimistic estimate of true error
- Test error rate
  - Used to evaluate hypothesis performance
  - Good estimate of true error (w/ enough test data)
- Validation error rate
  - Used to help choose $\mathcal{H}$ (e.g., set hyperparameters)
  - Somewhat optimistic estimate of true error

true distribution p^*; dataset X, Y

Error rate is also called risk

- True error rate = (true) ***risk*** — unknown

$$R(h) = \mathbb{E}$$

- Training error rate = ***empirical risk*** — can measure this

$$\hat{R}(h) = \mathbb{E}$$

write $\hat{\mathbb{E}}$ for …

Four classifiers

1. The *true classifier,* c^*: best answer but may be unachievable (and certainly unknown); only matters to define our setup
2. The (*true) risk minimizer* (best achievable, also unknown):

$$h^* = \underset{h \in \mathcal{H}}{\operatorname{argmin}} R(h)$$

3. The *empirical risk minimizer* (we can actually know this one):

$$\hat{h} = \underset{h \in \mathcal{H}}{\operatorname{argmin}} \hat{R}(h)$$

4. The *result of learning,* $\tilde{h}$*:* whatever our algorithm happens to return (we can know this one too, but we don't know much about it unless it is equal to $\hat{h}$)

Four classifiers

1. The *true classifier,* c^*: best answer but may be unachievable (and certainly unknown); only matters to define our setup
2. The (*true) risk minimizer* (best achievable, also unknown):

$$h^* = \underset{h \in \mathcal{H}}{\operatorname{argmin}} R(h)$$

3. The *empirical risk minimizer* (we can actually know this one):

$$\hat{h} = \underset{h \in \mathcal{H}}{\operatorname{argmin}} \hat{R}(h)$$

4. The *result of learning,* $\tilde{h}$*:* whatever our algorithm happens to return (we can know this one too, but we don't know much about it unless it is equal to $\hat{h}$)

ERM = learning method that picks hypothesis by minimizing empirical risk

i.e., $\tilde{h} = \hat{h}$

Overfitting

- Goal: we want to certify that either $\hat{h}$ or $\tilde{h}$ performs well
- Exact guarantee differs in the two cases:
  - $\hat{h}$: we know the optimizer got us the best hypothesis on the training set: we hope it performs nearly as well as h^*
  - $\tilde{h}$: no assumption about optimizer, so can't say as much: we hope it performs nearly as well as it *appears* to perform
- Both of these are guarantees about overfitting:
  - we chose a nearly-best hypothesis
  - we measured our hypothesis' performance well, even though we used an optimizer

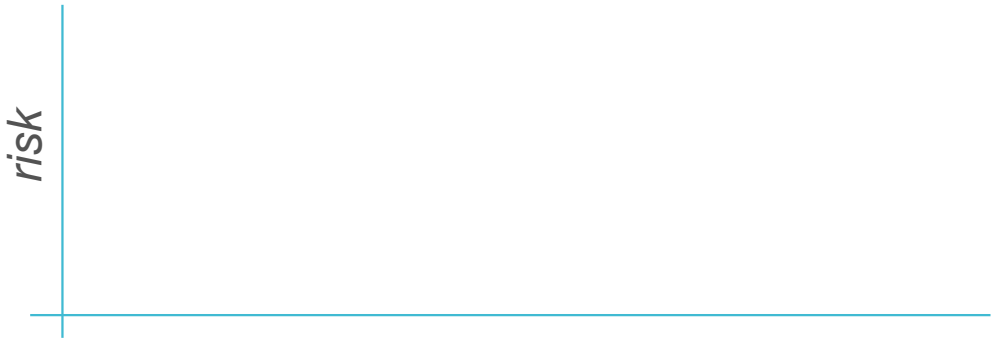
Probably approximately correct

why the name?

- PAC (Probably Approximately Correct) criterion:

$$P\left(\left|R(h) - \hat{R}(h)\right| \leq \epsilon, \forall h \in \mathcal{H}\right) \geq 1 - \delta$$

  - for some ϵ (difference between true and empirical risk) and δ (probability of failure)
- If we can guarantee PAC:
  - both $\hat{h}$ and $\tilde{h}$ will work about as well as we think they do
    - if all $h \in \mathcal{H}$ have true and empirical risk within ϵ then $\hat{h}$ and $\tilde{h}$ will have true and empirical risk within ϵ
  - $\hat{h}$ will be near-optimal: within 2ϵ of h^*



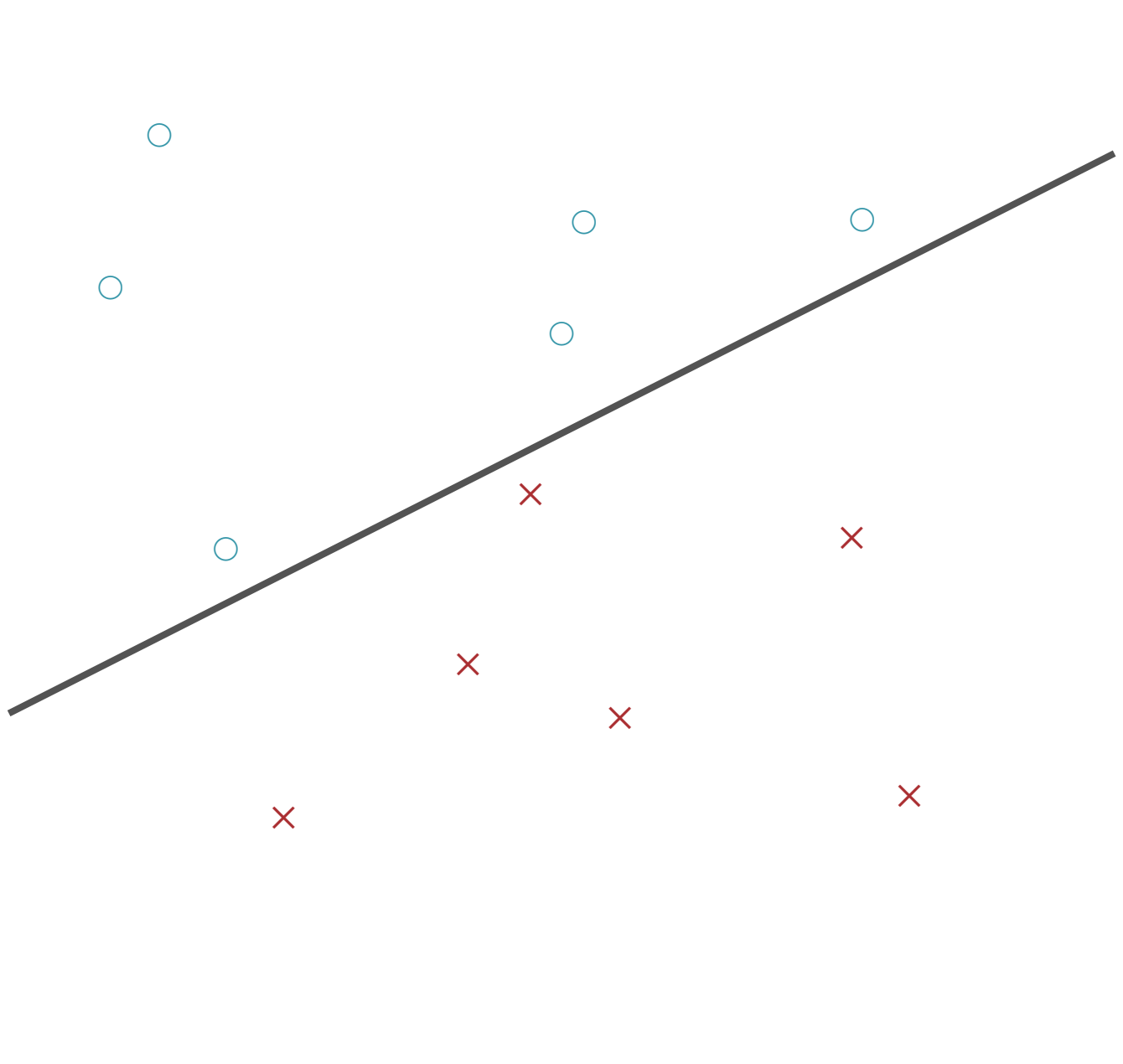
Sample complexity

- Fix a failure probability δ
- Our measurement accuracy ϵ will (hopefully) improve as the size N of our training set increases
- Def'n: ***sample complexity*** is the smallest N we need to satisfy the PAC criterion (as a function of $\mathcal{H}, \epsilon, \delta$)

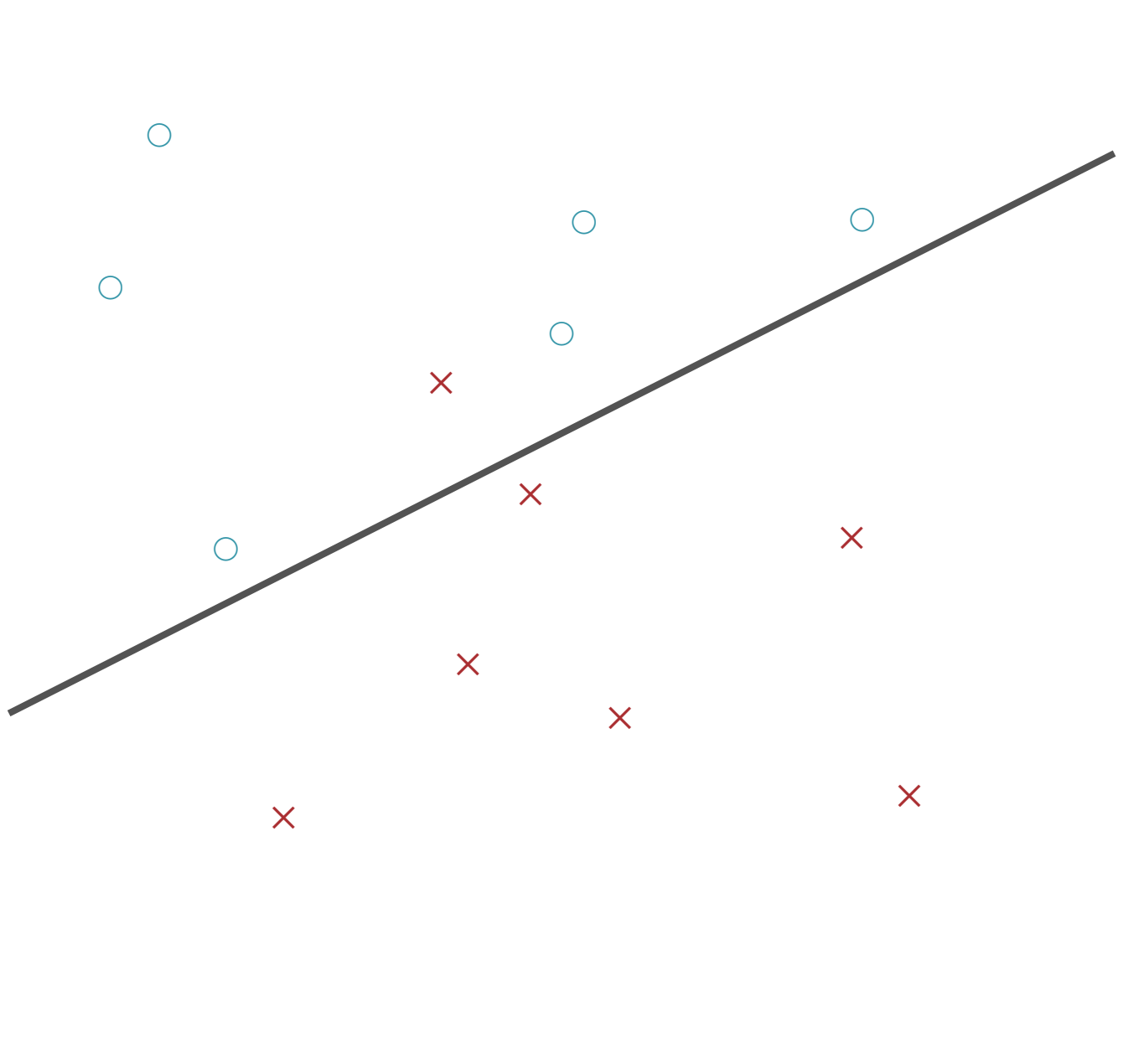
We will look at four cases

- Realizable vs. Agnostic
  - Realizable → $c^* \in \mathcal{H}$
  - Agnostic → c^* might or might not be in $\mathcal{H}$
- Finite vs. Infinite
  - Finite → $\left|\mathcal{H}\right| < \infty$
  - Infinite → $\left|\mathcal{H}\right| = \infty$

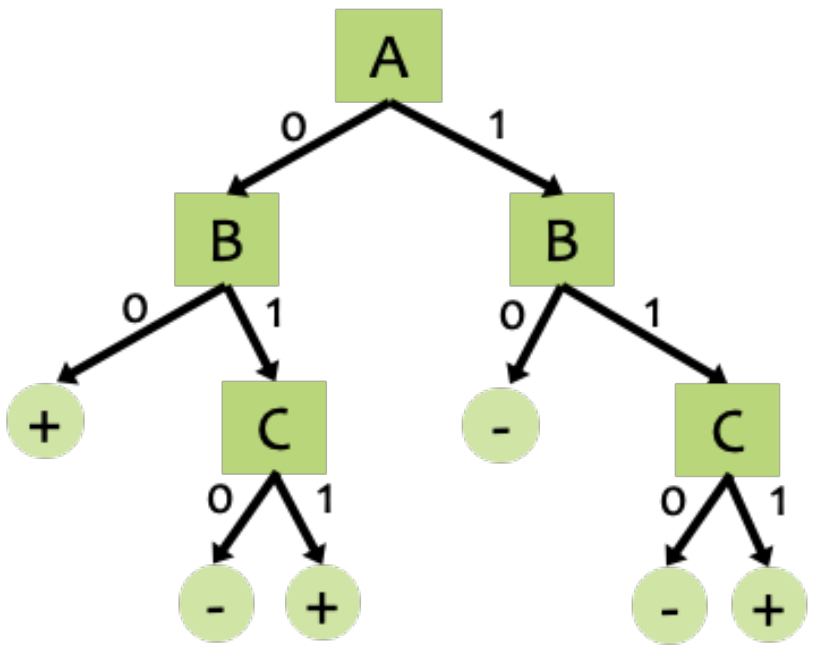
Realizable vs. agnostic

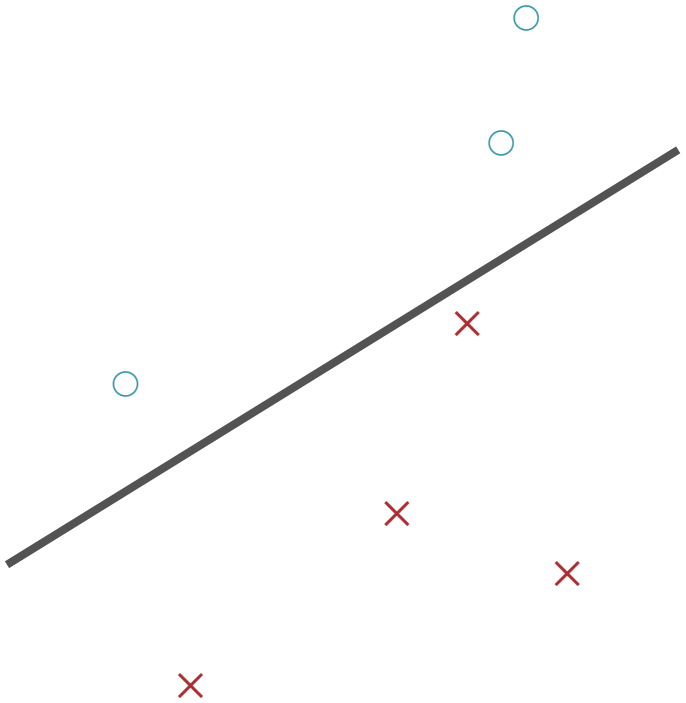


Realizable vs. agnostic



Finite vs. infinite





- Decision trees of bounded depth on discrete attributes vs. linear separators in 2D

Theorem: finite, realizable case

sometimes called ***Haussler bound***

only covers ERM

For a finite hypothesis set $\mathcal{H}$ s.t. $c^* \in \mathcal{H}$ and arbitrary distribution p^*, if the number of labelled training data points satisfies

$$N \geq \frac{1}{\epsilon}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right)$$

then with probability at least $1-\delta$, all $h \in \mathcal{H}$ with $\hat{R}(h) = 0$ have $R(h) \leq \epsilon$

We will prove this over the next few slides

Theorem: finite, realizable case

sometimes called ***Haussler bound***

only covers ERM

Bound is linear in $\frac{1}{\epsilon}$

Fo esis set $\mathscr{H}$ s.t. $c^* \in \mathscr{H}$ and arbitrary di he number of labelled training data points sa ies

$$N \geq \frac{1}{\epsilon}\left(\ln|\mathscr{H}| + \ln\frac{1}{\delta}\right)$$

then with probability at least $1-\delta$, all $h \in \mathscr{H}$ with $\hat{R}(h) = 0$ have $R(h) \leq \epsilon$

We will prove this over the next few slides

Theorem: finite, realizable case

sometimes called ***Haussler bound***

only covers ERM

Fo (and arbitrary

di training data

points sa ies

Bound is linear in $\frac{1}{\epsilon}$

but only logarithmic in $|\mathscr{H}|$ and $\frac{1}{\delta}$

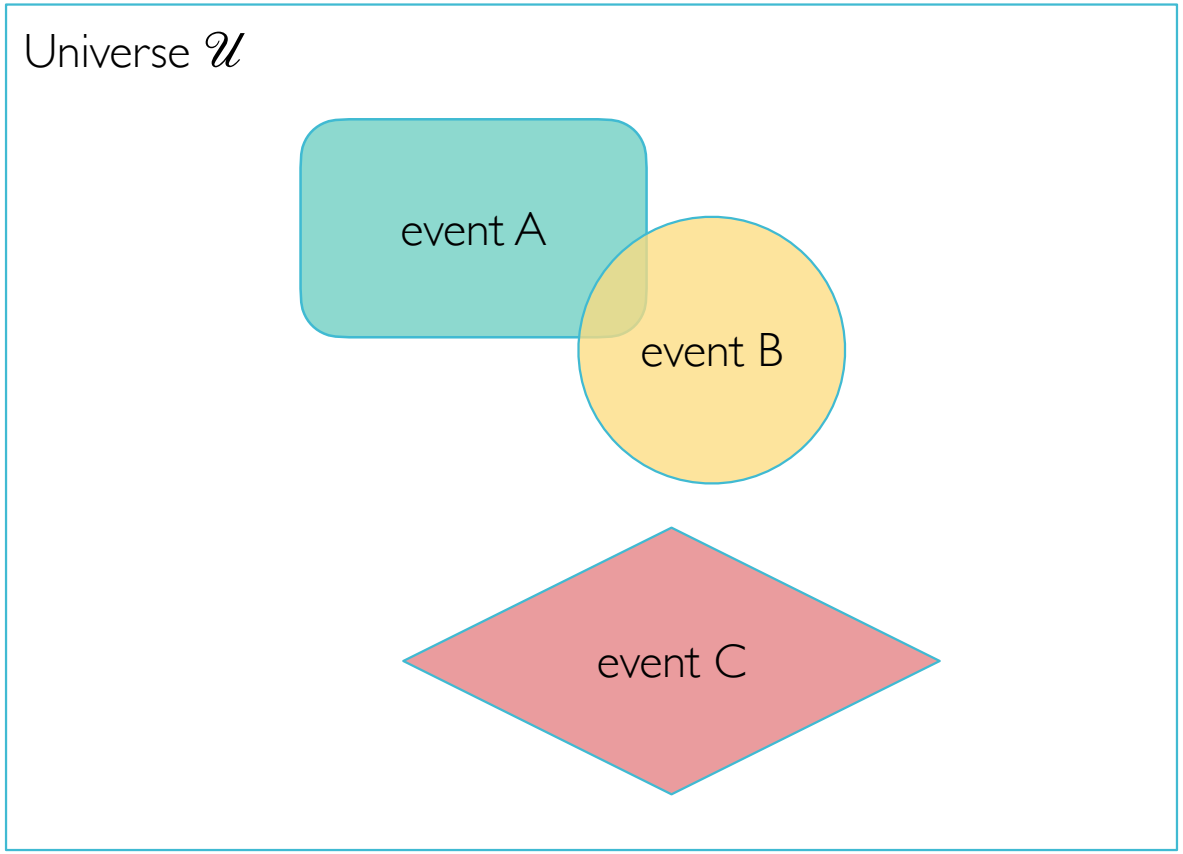
$$N \geq \frac{1}{\epsilon}\left(\ln|\mathscr{H}| + \ln\frac{1}{\delta}\right)$$

then with probability at least $1-\delta$, all $h \in \mathscr{H}$ with $\hat{R}(h) = 0$ have $R(h) \leq \epsilon$

We will prove this over the next few slides

Tools: union bound

used to combine bounds on individual events into a bound on all of them



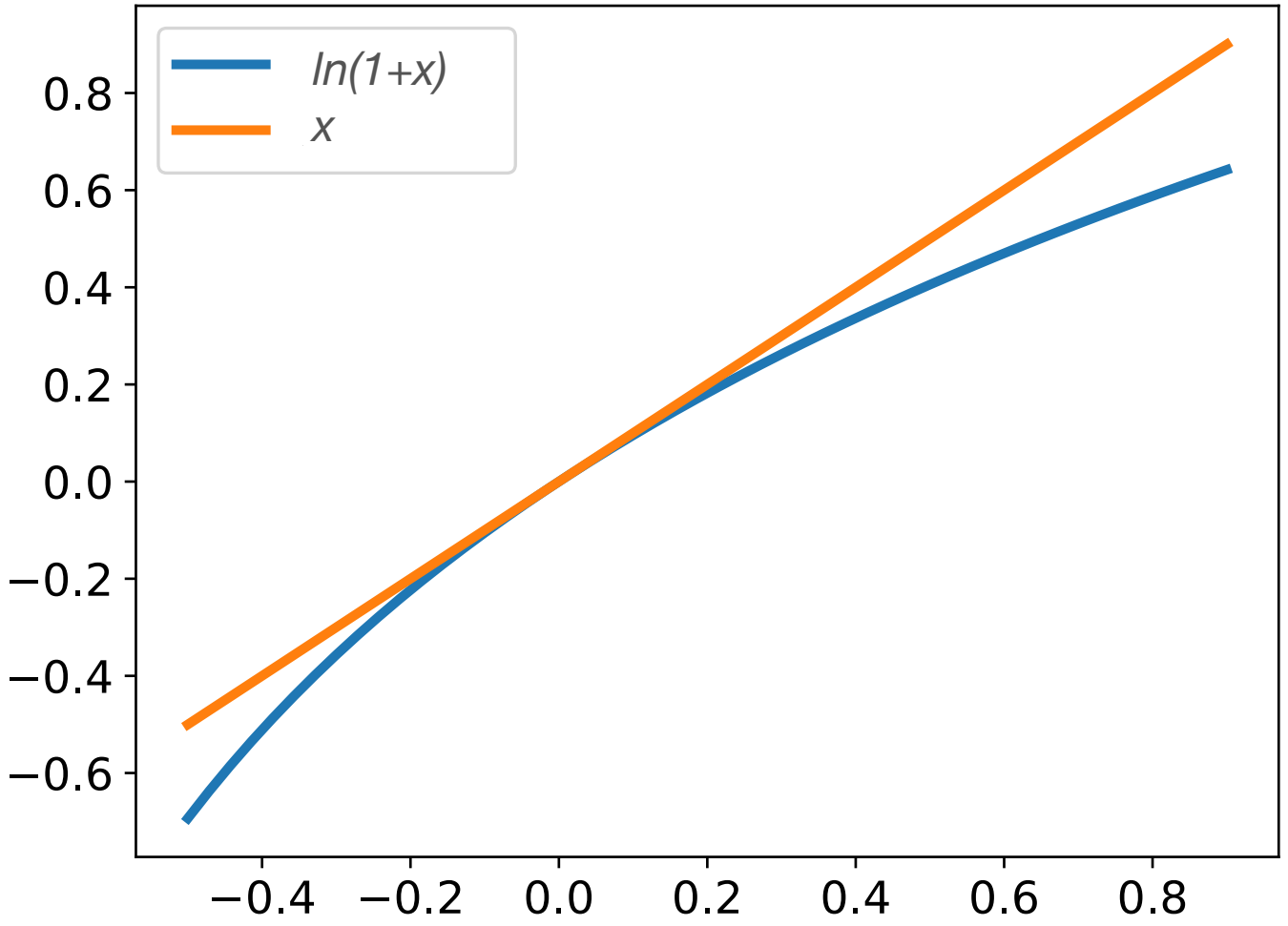
$$P(A \text{ or } B \text{ or } C) \leq P(A) + P(B) + P(C)$$

$$P(\text{some event happens}) \leq \text{sum of probabilities}$$

even holds for a countably infinite number of events

Tools: bound on ln

used to simplify bounds



$$\ln(1+x) \leq x$$

Notation for conclusion

- Theorem said: if

$$N \geq \frac{1}{\epsilon}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right)$$

then with probability at least $1-\delta$, all $h \in \mathcal{H}$ with $\hat{R}(h) = 0$ have $R(h) \leq \epsilon$

- Write E for bad event: $\exists h \in \mathcal{H}$ with $\hat{R}(h) = 0$, $R(h) > \epsilon$
- Theorem's conclusion is $P(E) < \delta$

Bound for one hypothesis

- Consider some h with $R(h) > \epsilon$. What's $P(\hat{R}(h) = 0)$?
  - bad event for a single hypothesis

Bound for k hypotheses

- Suppose there are k hypotheses with $R(h) > \epsilon$. What's $P(E)$, i.e., $P(\hat{R}(h_1) = 0 \text{ or } \hat{R}(h_2) = 0 \text{ or } \ldots)$?

Solve for N

we have

$$P(E) < |\mathcal{H}|(1-\epsilon)^N$$

Haussler bound: corollary

- For a finite hypothesis set $\mathcal{H}$ s.t. $c^* \in \mathcal{H}$, for an arbitrary distribution p^*, given a training data set S from p^* w/ $|S| = N$, all $h \in \mathcal{H}$ with $\hat{R}(h) = 0$ on S have

$$R(h) \leq \frac{1}{N}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right)$$

with probability at least $1 - \delta$.

Recall Theorem 1 said $N \geq \frac{1}{\epsilon}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right)$

Theorem: finite, agnostic case

tells us about both ERM and arbitrary $\tilde{h}$

- For a finite hypothesis set $\mathscr{H}$ and arbitrary distribution p^*, if the number of labelled training data points satisfies

$$N \geq \frac{1}{2\epsilon^2}\left(\ln|\mathscr{H}| + \ln\frac{1}{\delta}\right)$$

then w.p. at least $1-\delta$, all $h \in \mathscr{H}$ have $R(h) \leq \hat{R}(h) + \epsilon$

- Bound is inversely ***quadratic*** in ϵ, e.g., halving ϵ means we need four times as many labelled training data points

Theorem: finite, agnostic case

tells us about both ERM and arbitrary $\tilde{h}$

Main difference: quadratic

- …nesis set $\mathscr{H}$ and arbitrary distribution p^*, if the nu…er of labelled training data points satisfies

$$N \geq \frac{1}{2\epsilon^2}\left(\ln|\mathscr{H}| + \ln\frac{1}{\delta}\right)$$

then w.p. at least $1-\delta$, all $h \in \mathscr{H}$ have $R(h) \leq \hat{R}(h) + \epsilon$

- Bound is inversely ***quadratic*** in ϵ, e.g., halving ϵ means we need four times as many labelled training data points

Proof sketch

- To get $\hat{R}(h)$, need error count for h
  - distribution is binomial: N samples, probability $R(h)$
- Bad event is if $\hat{R}(h)$ is more than ϵ smaller than $R(h)$
  - this is a sum of binomial probabilities in the lower tail
  - some manipulation gets $P(\text{bad}) \leq \exp(-2\epsilon^2 N)$ for any h
- Just as before, union bound over all $h \in \mathscr{H}$ to find out how failure probability δ (i.e., probability of one or more of the bad events happening) relates to ϵ and N: it is $\delta \leq |\mathscr{H}| \exp(-2\epsilon^2 N)$
  - take log and rearrange: $2\epsilon^2 N \leq \ln|\mathscr{H}| + \ln\frac{1}{\delta}$

Finite, agnostic case: corollary

Again, by making the bound tight and solving for N

- For a finite hypothesis set $\mathcal{H}$ and arbitrary distribution p^*, given a training data set S s.t. $|S| = N$, all $h \in \mathcal{H}$ have

$$R(h) \leq \hat{R}(h) + \sqrt{\frac{1}{2N}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right)}$$

with probability at least $1 - \delta$.